



第 11 课 - 字典的终极大法

● dir()

- ✧ 功能：查看字典内置所有方法
- ✧ 语法：dir({}) 或 dir(dict)

● clear()

- ✧ 功能：用于清空字典中所有的 key-value 对
- ✧ 说明：对一个字典执行 clear() 方法之后，该字典就会变成一个空字典
- ✧ 语法：字典名.clear()
- ✧ 案例：

```
01. cars = {'BMW': 8.5, 'BENS': 8.3, 'AUDI': 7.9}
02. print(cars)
>>> {'BMW': 8.5, 'BENS': 8.3, 'AUDI': 7.9}
03. cars.clear() # 清空cars所有key-value对
04. print(cars)
>>> {} # cars字典变成了空字典
```



● get()

- ✧ 功能：根据 key 来获取 value
- ✧ 说明：key 不存在时，结果为 None
- ✧ 语法：字典名.get(key)
- ✧ 案例：（右图）

```
01. cars = {'BMW': 8.5, 'BENS': 8.3, 'AUDI': 7.9}
02. # 获取'BMW'对应的value
03. print(cars.get('BMW'))
>>> 8.5
04. print(cars.get('AUTO'))
>>> None
05. print(cars['AUTO'])
>>> KeyError
```

● update()

✧ 功能：使用一个字典所包含的键值对来更新已有的字典

✧ 说明：

- ① 如果 key 存在，value 会被更新
- ② 如果 key 不存在，key-value 会被新增
- ③ 原字典中其他的 key-value 不变

✧ 语法：字典名.update(新字典)

✧ 案例：

```
01. cars = {'BMW': 8.5, 'BENS': 8.3, 'AUDI': 7.9}
02. cars.update({'BMW': 4.5, 'PORSCH': 9.3})
03. print(cars)
>>> {'BMW': 4.5, 'BENS': 8.3, 'AUDI': 7.9, 'PORSCH': 9.3}
```

● pop()

✧ 功能：获取指定 key 对应的 value，并删除这个 key-value 对

✧ 语法：字典名.pop(key)

✧ 案例：

```
01. cars = {'BMW': 8.5, 'BENS': 8.3, 'AUDI': 7.9}
02. print(cars.pop('AUDI'))
>>> 7.9
03. print(cars)
>>> {'BMW': 8.5, 'BENS': 8.3}
```



● popitem()

✧ 功能：弹出底层存储的最后一个 key-value 对

✧ 语法：字典名.popitem()

✧ 案例：

```
01. cars = {'BMW': 8.5, 'BENS': 8.3, 'AUDI': 7.9}
02. print(cars.popitem())
>>> ('AUDI', 7.9) # 元组类型
03. print(cars)
>>> {'BMW': 8.5, 'BENS': 8.3}
```



● setdefault()

✧ 功能：设置默认值

✧ 说明：

① 当 key 不存在时，会先为这个 key 设置一个默认的 value，然后再根据该 key 获取 value。

② 当 key 存在时，则会根据 key 来获取对应 value

✧ 语法：字典名.setdefault()

✧ 案例：

```
01. cars = {'BMW': 8.5, 'BENS': 8.3, 'AUDI': 7.9}
02. # key不存在，则新增key-value对
03. print(cars.setdefault('PORSCE', 9.2))
>>> 9.2
04. print(cars)
>>> {'BMW': 8.5, 'BENS': 8.3, 'AUDI': 7.9, 'PORSCE': 9.2}
05. # key存在，则不会修改dict内容
06. print(cars.setdefault('BMW', 3.4))
>>> 8.5
07. print(cars)
>>> {'BMW': 8.5, 'BENS': 8.3, 'AUDI': 7.9, 'PORSCE': 9.2}
```



● fromkeys()

✧ 功能：创建字典

✧ 说明：

- 以某序列中的元素作为字典的 key，value 默认为 None
- 也可以自行指定 value 的值

✧ 语法：dict.fromkeys(序列)

✧ 案例：

```
01. seq=('name','age','job','salary') #元组类型
02. d=dict.fromkeys(seq)
03. print(d)
>>> {'name':None,'age':None,'job':None,'salary':None}
```

```
01. seq=('name','age','job','salary') #元组类型
02. d=dict.fromkeys(seq,20000)
03. print(d)
>>> {'name':20000,'age':20000,'job':20000,'salary':20000}
```

● 字典内置方法总结

方法	功能
dir()	查看字典内置所有方法
clear()	用于清空字典中所有的 key-value 对
get()	根据 key 来获取 value
update()	使用一个字典所包含的键值对来更新已有的字典
pop()	获取指定 key 对应的 value，并删除这个 key-value 对
popitem()	弹出底层存储的最后一个 key-value 对
setdefault()	设置默认值
fromkeys()	创建字典

● 字典内置方法复习

方法	功能
<code>dict.keys()</code>	获取字典所有key
<code>dict.values()</code>	获取字典所有value
<code>dict.items()</code>	获取字典所有(key,value)的元组
<code>for...in dict:</code>	遍历字典(key)
<code>if...in dict:</code>	判断字典(key)中是否含有...
<code>len(dict)</code>	字典长度
<code>dict()</code>	创建字典

● 列表与字典对比

◇ 整体对比

比较项	列表	字典
类型名称（创建方法）	list	dict
符号	[]	{}
是否可变	是	是
是否有序	是	否
是否有索引	是	否，有key
每组元素之间分隔符	逗号	逗号
对所存元素的格式要求	无	key:value
对所存元素的值的要求	无	key为不可变数据类型
所存元素是否可重复	是	key不可重复，value可重复
元素查找速度（拓展）	非常慢	非常快
增/删元素的速度（拓展）	尾部新增快，其他位置慢	快

◇ 功能对比

比较项	列表	字典
访问/查找其中某元素	<code>list[index]</code>	<code>dict[key]</code>
修改元素	<code>list[index] = a</code>	<code>dict[key] = value</code>
添加元素	<code>list.append()</code>	<code>dict[key] = value</code>
删除元素	<code>del/pop()/remove()</code>	<code>del dict[key]</code>
遍历	<code>for i in list:</code>	<code>for k in dict.keys()</code>
		<code>for v in dict.values()</code>
		<code>for k,v in dict.items()</code>